

Flutter Developer Assessment

News & Weather Hub — Home Assignment

Objective

Build a production-ready Flutter application that combines weather information and news updates. We're less interested in a pixel-perfect UI than in how you structure, test, and reason about a real-world app. Your submission should demonstrate:

- Architecture
- State management
- API integration
- Local persistence
- Responsive, accessible UI
- Code quality, readability, and maintainability

State Management

You're free to use any of the following — choose whatever you're most comfortable building a scalable, maintainable app with:

- BLoC
- Cubit
- GetX
- Riverpod
- Provider

APIs

Weather — Open-Meteo

<https://api.open-meteo.com/v1/forecast>

- No authentication required
- Completely free, generous rate limits for non-commercial use
- Required params: latitude, longitude, current_weather=true, daily=temperature_2m_max,temperature_2m_min,weathercode, timezone=auto

Important: the forecast endpoint only accepts latitude/longitude — it has no "city name" parameter. To support "search weather by city name," first call Open-Meteo's free Geocoding API to resolve the city to coordinates, then pass those coordinates to the forecast endpoint. Geocoding endpoint: <https://geocoding-api.open-meteo.com/v1/search?name={city}> (no auth required, returns a list of matching locations with lat/lon).

News — NewsAPI.org

<https://newsapi.org>

- Sign up for a free API key at newsapi.org/register
- Useful endpoints: `/v2/top-headlines` (supports country & category filters), `/v2/everything`

Note: NewsAPI's free "Developer" tier is licensed for development/testing use only (not production traffic) and has a daily request cap. This is perfectly fine for this assignment — just don't be surprised by rate limits, and treat your API key as a dev-only credential. Mention in your README which plan/limits you hit, if any.

API Key Handling

- Store the NewsAPI key using environment variables (e.g. `--dart-define`, a non-committed `.env` file with `flutter_dotenv`, or platform-specific config).
- Do NOT commit API keys to the repository. Include a `.env.example` or equivalent with placeholder values, and add the real file to `.gitignore`.
- Open-Meteo needs no key, so this requirement applies to the News API only.

Implementation Constraints

- Write your own networking/repository layer for both APIs (using `http` or `dio` is fine). Do not use an all-in-one wrapper package that fetches/parses weather or news for you — we want to see how you structure the data layer.
- Local persistence (`Hive`/`SQLite`/`Isar`) must be wired by you, not auto-generated boilerplate left untouched — you should be able to explain your schema.
- All packages used must be free and open-source (`pub.dev`). No paid SDKs, no services requiring a credit card.

Required Features

Screen 1 — Dashboard

Weather Section

Display:

- Current temperature
- Weather condition
- Humidity
- Wind speed
- City name

User should be able to:

- Use current device location, or
- Search weather by city name (via the geocoding step described above)
- City search must be debounced (e.g. `~400–500ms` after the user stops typing) and cancel any in-flight request when the user types again — avoid firing a request per keystroke or showing stale results from an outdated query.

Forecast Section

- Next 5 days forecast

- Horizontal scrolling list

News Section

Display:

- Latest headlines
- Thumbnail image
- Source name
- Published date
- Load articles with pagination / infinite scroll as the user scrolls down (don't just fetch a fixed batch of 10 once) — at least 20 articles should be reachable by scrolling.

Resilience

- If the News or Weather API call fails (no internet, timeout, or non-200), retry automatically with a short backoff (e.g. 1–2 attempts) before showing an error.
- Cache the last successful weather and news response locally. If a refresh fails, show the cached data with a small "showing offline data from [time]" indicator instead of a blank error screen.

Refresh

- Support pull-to-refresh on the dashboard (refreshing both weather and news).

Screen 2 — News Details

Display:

- Article image
- Title
- Description
- Source
- Published date

Actions:

- Open full article in the browser
- Share article

Screen 3 — Bookmarks

Users should be able to:

- Save an article
- Remove an article
- View all bookmarked articles

Requirements:

- Persist data locally and survive app restarts
- Use Hive, SQLite, or Isar (candidate's choice)

Screen 4 — Settings

Theme Settings

- Light mode
- Dark mode
- Persist the selection locally

Weather Settings

- Allow the user to save a default city

About Section

- App version
- Build number

Error Handling & Edge Cases (evaluated)

Please handle, at minimum:

- No internet connection
- API failures / non-200 responses (including NewsAPI rate-limit responses)
- Location permission denied
- City not found (geocoding returns no results)
- Empty states (no bookmarks, no news returned)

Design Decisions Write-Up

In your README, answer the following in your own words (a few sentences each is enough):

1. What happens in your app if both the Weather and News APIs fail at the same time? Walk through it screen by screen.
2. Why did you choose your specific local persistence option (Hive/SQLite/Isar), and what would change if bookmarks needed to sync across a user's devices?
3. If you had one more day, what's the first thing you'd refactor or add, and why?

Submission Guidelines

Item	Details
Repository	Push your code to a public/private GitHub repo and APK
README	Include setup steps, how to add the NewsAPI key, architecture overview, your Design Decisions answers (above), and how to run the tests.
Questions	Reach out to same email if any requirement is unclear.